



Репозиторий содержит краткую документацию в примерах по основам синтаксиса JavaScript для Node.js на **русском языке**.

Навигация:

- [Переменные](#)
- [Типы данных](#)
- [Функции](#)
- [Условия](#)
- [Обработка ошибок](#)
- [Массивы](#)
- [Объекты](#)
- [Циклы](#)
- [Асинхронные операции](#)
- [Регулярные выражения](#)
- [Математические вычисления](#)
- [Express](#)
- [Axios](#)

- [Fetch](#)
- [Cheerio](#)
- [Puppeteer](#)

Переменные

Переменная `var` поддерживает любую область видимости и ее возможно объявить повторно (считается устаревшим методом и рекомендуется не использовать).

```
console.log(varVariable) // Uncaught ReferenceError: varVariable is not defined (не определена)
if (true) {
    var varVariable = true
}
console.log(varVariable) // true
var varVariable = 123
console.log(varVariable) // 123
```

Переменная `let` позволяет изменять содержимое с другим типом данных (в отличии от `const`), но ограничена областью видимости (в отличии от `var`).

```
console.log(letVariable) // Uncaught ReferenceError: letVariable is not defined
if (true) {
    let letVariable = 'letVariable is defined'
}
console.log(letVariable) // Uncaught ReferenceError: letVariable is not defined
let letVariable = 'string'
console.log(letVariable)
let letVariable = 'string' // Uncaught SyntaxError: Identifier 'letVariable' has already been declared
let letVariable = 123
console.log(letVariable)
```

Переменная константа `const` требует обязательной инициализации во время объявления, и не позволяет изменять значение переменной после ее объявления.

```
const constVariable = 'string'
constVariable = '123' // Uncaught TypeError: Assignment to constant variable (ошибка присвоения)
```

Типы данных

```
typeof 32 // number
typeof 3.2 // number
typeof 123n // bigint
typeof 'text' // string
typeof true // boolean
typeof false // boolean
typeof null // object
typeof { key: 'value' } // object
typeof [1, 2, 3] // object
typeof function() {} // function
typeof Symbol('id') // symbol
```

Преобразовать аргумент в число. Возвращает NaN , если преобразование невозможно.

```
Number("123") // 123
Number("123.45") // 123.45
Number("abc") // NaN
Number(undefined) // NaN
Number(true) // 1
Number(false) // 0
Number(null) // 0
```

Преобразовать строку в целое число. Прекращает чтение, как только встречает нецифровой символ.

```
parseInt("123abc") // 123
parseInt("12.34") // 12
parseInt("abc123") // NaN
```

Преобразовать строку в число с плавающей точкой.

```
parseFloat("123.45abc") // 123.45
parseFloat("abc123.45") // NaN
```

Преобразовать значение в строку.

```
(123).toString() // "123"
(true).toString() // "true"
String(123)      // "123"
String(true)     // "true"
String(false)    // "false"
String(null)     // "null"
String(undefined) // "undefined"
```

Преобразовать значение в булево (логическое) значение. Все значения, кроме `0`, `null`, `NaN`, `undefined` и пустой строки, преобразуются в `true`.

```
Boolean(123)      // true
Boolean("Hello")  // true
Boolean(0)        // false
!0               // true
!!0              // false
Boolean(null)     // false
Boolean(NaN)      // false
Boolean(undefined) // false
Boolean("")       // false
```

Функции

```
function sum (param1, param2) {
  console.log(param1*param2)
}
```

```
sum(2,2) // 4
```

```
function getRandomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min
}
```

```
getRandomInt(1,100) // Например, 44
```

Условия

```
function functionName (paramName) {  
  if (paramName === 0) {  
    console.log(`${paramName} равно 0`)  
  }  
  else if (paramName < 10) {  
    console.log(`${paramName} меньше 10`)  
  }  
  else if (paramName >= 10) {  
    console.log(`${paramName} больше или равно 10`)  
  }  
  else {  
    console.log(`Переданное значение - ${paramName} - не подходит под заданные условия`)  
  }  
}
```

```
functionName(0)      // 0 равно 0  
functionName(5)      // 5 меньше 10  
functionName(15)     // 15 больше или равно 10  
functionName('test') // Переданное значение - test - не подходит под заданные условия
```

Однострочный формат условия `?:`, который подходит для использования в теле переменной

```
let input = 1  
input === 1 ? true : false // true  
input = 2  
input === 1 ? true : false // false
```

Проверить одно значение сразу на большое количество условий с помощью конструкции `switch`

```
function getDayOfWeek(day) {  
  switch (day) {  
    case 1:  
      return 'Понедельник'  
    case 2:  
      return 'Вторник'  
    case 3:  
      return 'Среда'  
    case 4:  
      return 'Четверг'  
    case 5:  
      return 'Пятница'  
    case 6:  
      return 'Суббота'  
    case 7:  
      return 'Воскресенье'  
    default:  
      return 'Неправильно задан параметр'  
  }  
}
```

```
console.log(getDayOfWeek(1)) // Понедельник  
console.log(getDayOfWeek(5)) // Пятница  
console.log(getDayOfWeek(7)) // Воскресенье  
console.log(getDayOfWeek(8)) // Неправильно задан параметр
```

Обработка ошибок

```
function errorTest(a, b) {  
  // Блок кода, который будет выполняться до тех пор, пока не возникнет ошибка  
  try {  
    if (b === 0) {  
      throw new Error("на ноль делить нельзя")  
    }  
    console.log(a / b) // выполняется, если ошибок нет  
  }  
  // Блок кода, который будет выполнен, если в блоке try возникнет ошибка  
  catch (error) {  
    console.log(`Ошибка: ${error.message}`) // Обработка ошибки  
  }  
  // Блок кода, который выполняется в любом случае  
  finally {  
    console.log("Блок finally выполняется всегда") // Всегда выполняется  
  }  
}  
  
errorTest(10, 2)  
// 5  
// Блок finally выполняется всегда  
  
errorTest(10, 0)  
// Ошибка: на ноль делить нельзя  
// Блок finally выполняется всегда
```

Массивы

Классический массив и его методы.

```

const array = [3, 2, 'string']
Array.isArray(array) // Проверка, является ли переменная массивом (true)
array[2] = 1 // Изменить содержимое элемента в массиве по индексу
array.sort() // сортировка по умолчанию: [ 1, 2, 3 ]
array.reverse() // Поменять порядок следования: [ 3, 2, 1 ]
array[0] + array[1] + array[2] // 6
array[3] // undefined (не определено)
array[3] = 4 // Добавить новый элемент в массив по индексу
array.push(5) // Добавить новый элемент с конца
array[4] // 5
array.slice(3,5) // Вывести содержимое массива с 4-го по 5-й индекс: [ 4, 5 ]
array.unshift(0) // Добавить элемент(ы) (через запятую) в начало массива
array.shift() // Удалить первый элемент из массива
array.pop() // Удалить последний элемент из массива
array[array.length-1] // Вывести содержимое последнего элемента в массиве
array.indexOf(1) // Выводит индекс первого вхождения элемента в массив, или -1, если элемент не

```

Вложенный массив, а также методы фильтрации и объединения:

```

const data = [
  { id: 1, name: 'red' },
  { id: 2, name: 'blue' },
]

data.push({id: 3, name: 'green'}) // Добавить новый элемент в массив

const nameArray = data.map(item => item.name) // Пересобрать новый массив

const filterArray = nameArray.filter(item => item.length >= 4) // отфильтровать содержимое массива
// [ 'blue', 'green' ]

[...nameArray, ...filterArray] // объединить два массива
// [ 'red', 'blue', 'green', 'blue', 'green' ]

const idArray = data.map(item => item.id)
// Метод reduce выполняет функцию для каждого элемента массива, чтобы получить одно итоговое значение
idArray.reduce((accumulator, current) => accumulator + current, 0) // 3

```

Объекты

Преобразовать объект в массив и наоборот:


```

const obj = { a: 1, b: 2, c: 3 }
obj.b // 2
let keys = Object.keys(obj)      // [ 'a', 'b', 'c' ]
let values = Object.values(obj)  // [ 1, 2, 3 ]
const arr = Object.entries(obj)  // [ ["a", 1], ["b", 2], ["c", 3] ]
arr[1][1] // 2
Object.fromEntries(arr)          // { a: 1, b: 2, c: 3 }

```

Объект представляет из себя список пар (ключ-значение), разделенного запятыми. Используя переменную `const` при объявлении объектов, возможно изменять содержимое дочерних элементов.

```

const box = {
  height: 8,
  width: 40,
  scrollable: true,
  style: {
    fg: 'white',
    bg: 'black'
  }
}

box.height // 8
box.style // { fg: 'white', bg: 'black' }
box.style.fg // white

box.style.fg = 'blue'
box.style.fg // blue

```

Объекты и вложенные массивы JavaScript могут содержать дочерние массивы внутри `[]` и вложенные объекты внутри `{}`.

```

const obj = [
  'JavaScript',
  2024,
  [
    'Express',
    'Axios',
  ],
  {
    name: 'Alex',
    age: 29,
    num: [1, 'string', {}],
    test: {}
  }
]

```

Конвертация в JSON :

```

const jsonString = JSON.stringify(obj) // Конвертация из объекта JavaScript в JSON
const obj = JSON.parse(jsonString)    // Конвертация из JSON в JavaScript

```

Методы объекта назначаются через функции

```

const obj = {
  default: 10,
  get() {
    return this.default
  },
  plus(num) {
    if (isNaN(num)) {return}
    this.default += num
  },
  minus(num) {
    if (isNaN(num)) {return}
    this.default -= num
  }
}

```

```

obj.minus(1)
obj.get() // 9
obj.plus(2)
obj.get() // 11

```

Оператор `return` используется для выхода из функции (т.е. последующий код не читается), который возвращает значение указанное после ключевого слова.

Циклы

Примеры циклов взяты из проекта [multranslate](#), для проверки всех строк в массиве и увеличения количества видимых строк с учетом `autowrap`. Имеется две реальных строки, необходимо узнать количество виртуальных строк с учетом длины символов в строке. Например, если максимальная длина одной строки составляет, 36, то на одну реальную строку в 92 символа приходится дополнительно еще 2 виртуальных. Количество найденных виртуальных строк прибавляется к изначально зафиксированному значению количества всех реальных строк.

```
// На входе 2 строки, разделенные символом \r
const text = "Первая строка\rВторая очень длинная строка, которое будет превышать максимальное :

// Фиксируем текущее максимальное количество строк и длину символов в строке с учетом размеров
const maxLines = box.height - 2 // 6
const maxChars = box.width - 4 // 36

// Разбиваем текст на массив из строк
const bufferLine = text.split('\r')
// Забираем реальное количество строк (2)
let viewLines = bufferLine.length

// Вывести количество строк в каждой строке массива
bufferLine.map(item => item.length) // [ 13, 92 ]
```

Классический цикл `for` итерирует числами с типом данных `number` (`int` / `integer`). С каждой интерацией объявленное значение (`let i = 0`) увеличивается на заданное количество (`i++` - на единицу), цикл завершается в случае успешного соблюдения условия (`i < bufferLine.length`).

```

for (let i = 0; i < bufferLine.length; i++) {
  if (bufferLine[i].length > maxChars) {
    // Добавляем одну виртуальную строку без учета остатка символов
    viewLines++
  }
}
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 3
// Уменьшаем значение на 1, для проверки в других циклах
viewLines--

for (let i = 0; i < bufferLine.length; i++) {
  if (bufferLine[i].length > maxChars) {
    // Добавляем все виртуальные строки, с округлением в меньшую сторону
    viewLines += Math.floor(bufferLine[i].length / maxChars)
  }
}
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4
viewLines -= 2

```

Цикл `for...in` итерирует по индексам массива, с типом данных `string`. Сколько элементов в массиве (`bufferLine.length`), столько и будет индексов (отсчет начинается с нуля).

```

for (let index in bufferLine) {
  if (bufferLine[Number(index)].length > maxChars) {
    viewLines += Math.floor(bufferLine[i].length / maxChars)
  }
}
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4
viewLines -= 2

```

Цикл `for...of` итерирует по элементам массива, с уникальным типом данных каждого элемента в массиве.

```

const array = [1, 'string', {}]
for (let arr of array) {
    console.log(typeof arr)
}
// number
// string
// object

for (let line of bufferLine) {
    console.log(typeof line)
    if (line.length > maxChars) {
        viewLines += Math.floor(bufferLine[1].length / maxChars)
    }
}
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4
viewLines -= 2

```

Метод массива `forEach` выполняет указанную функцию для каждого элемента в массиве.

```

bufferLine.forEach(line => {
    if (line.length > maxChars) {
        viewLines += Math.floor(bufferLine[1].length / maxChars)
    }
})
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4
viewLines -= 2

```

Цикл `while` Выполняет тело цикла `{}` до тех пор, пока условие истинно

```

let i = 0
while (i < bufferLine.length) {
    if (bufferLine[i].length > maxChars) {
        viewLines += Math.floor(bufferLine[1].length / maxChars)
    }
    i++
}
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4
viewLines -= 2

```

Цикл `do..while` сначала выполняет тело цикла, а затем проверяет условие, это гарантирует, что интерация будет выполнена как минимум один раз.

```

i = 0
do {
  if (bufferLine[i].length > maxChars) {
    viewLines += Math.floor(bufferLine[1].length / maxChars)
  }
  i++
} while (i < bufferLine.length)
console.log(`${maxLines} ${maxChars} ${viewLines}`) // 6 36 4

```

Операторы:

- `continue` - прерывает текущую итерацию, для продолжения цикла с следующим значением
- `break` - прерывает и завершает цикл

```

let arr = [0, 1, 2, 4]
for (const value of arr) {
  console.log(`Начало ${value}-й итерации`)
  if (value === 1) {
    console.log('Пропустить оставшуюся часть кода и перейти к следующей итерации')
    continue
  } else if (value === 2) {
    console.log('Полностью выйти из цикла')
    break
  }
  console.log(`Конец ${value}-й итерации`)
}

```

```

// Начало 0-й итерации
// Конец 0-й итерации
// Начало 1-й итерации
// Пропустить оставшуюся часть кода и перейти к следующей итерации
// Начало 2-й итерации
// Полностью выйти из цикла

```

Асинхронные операции

`Promise` (промис) — это объект, который используется для обработки асинхронных операций, позволяя работать с результатами, когда они станут доступны, не блокируя основной поток выполнения. Он может находиться в одном из трех состояний:

- Pending (Ожидание): Операция только отправлена на выполнение или еще выполняется.
- Fulfilled (Выполнен): Операция завершилась успешно.
- Rejected (Отклонен): Операция завершилась с ошибкой.

С помощью ключевых слов `resolve` (разрешить/успех) и `reject` (отклонить/ошибка) производится управление возвращаемым результатом выполнения.

```
let testPromise = new Promise((resolve, reject) => {  
  if (true) {  
    resolve("Операция выполнена успешно")  
  } else {  
    reject("Ошибка выполнения операции")  
  }  
})  
  
testPromise.then(result => console.log(result)).catch(error => console.error(error))
```

Метод `then` используется для обработки успешного выполнения промиса (в состоянии `fulfilled`).

Метод `catch` используется для обработки ошибок или отказов промиса (в состоянии `rejected`).

`async` — это ключевое слово, которое делает функцию асинхронной и позволяет использовать `await`, чтобы приостановить выполнение до тех пор, пока все промисы не будут выполнены.

`await` — это ключевое слово, которое используется внутри асинхронной функции (`async`). Оно заставляет ждать выполнения промиса и возвращает его результат.

```

// Импортируем функцию exec из модуля child_process, которая позволяет запускать команды операц
const { exec } = require('child_process')

// Основная функция выполнения команды ping в промис
function pingHost(host) {
  return new Promise((resolve) => {
    // Выполняем команду ping для указанного хоста с timeout 50 мс
    exec(`ping -n 1 -w 50 ${host}`, (error) => {
      // Если нет ошибки, значит хост отвечает (alive: true)
      if (!error) {
        resolve({ host, alive: true })
      }
      // Если есть ошибка, хост не отвечает (alive: false)
      else {
        resolve({ host, alive: false })
      }
    })
  })
}

// Асинхронная функция создания промисов и получения результатов
async function pingSubnet(subnet) {
  // Массив для хранения промисов
  const promises = []
  // Генерация и пинг каждого IP-адреса в диапазоне от 1 до 254
  for (let i = 1; i <= 254; i++) {
    // Формируем IP-адрес, заменяя последний октет подсети
    const host = `${subnet.split('.').slice(0,3).join('.')}.${i}`
    // Добавляем промис в массив для выполнения пинга в фоне и продолжения интерации
    promises.push(pingHost(host)) // Promise { <pending> }
  }
  // Ждем завершения выполнения всех промисов
  const results = await Promise.all(promises)
  results.forEach(result => {
    if (result.alive) {
      console.log(`+++ ${result.host}`)
    } else {
      console.log(`- ${result.host}`)
    }
  })
}

```



```
pingSubnet('192.168.3.0')
```

Использовать внешнюю библиотеку [ping](#): `npm install ping`

```
const ping = require('ping')

// Асинхронная функция отправки команды ping через библиотеку
async function pingHost(host) {
  try {
    const res = await ping.promise.probe(host, { timeout: 1 })
    return { host, alive: res.alive }
  } catch (error) {
    return { host, alive: false }
  }
}

// Функция возврата промисов вручную без async
function pingHost(host) {
  return ping.promise.probe(host, { timeout: 1 })
    .then(res => {
      return { host, alive: res.alive }
    })
    .catch(error => {
      return { host, alive: false }
    })
}

async function pingSubnet(subnet) {
  const promises = []
  for (let i = 1; i <= 254; i++) {
    const host = `${subnet.split('.').slice(0,3).join('.')}.${i}`
    promises.push(pingHost(host))
  }
  const results = await Promise.all(promises)
  results.forEach(result => {
    if (result.alive) {
      console.log(`+++ ${result.host}`)
    } else {
      console.log(`- ${result.host}`)
    }
  })
}

pingSubnet('192.168.3.0')
```

`await Promise.all()` - дожидается успешного выполнения всех запросов.

`await Promise.allSettled()` - дожидается выполнения всех запросов не зависимо от успеха (возвращает статус и результат).

`await Promise.race()` - дожидается первого успешного выполнения, что бы получить результат от него не зависимо от его успеха.

Регулярные выражения

Преобразовать строку в массив из букв (`char`).

```
let line = "javascript"
let arr = Array.from(str) // ['j', 'a', 'v', 'a', 's', 'c', 'r', 'i', 'p', 't']
```

Метод `split()` используется для преобразования строки в массив.

```
line = "1,2,3,4,5" // '1,2,3,4,5'
arr = line.split(",") // [ '1', '2', '3', '4', '5' ]
```

Метод `join()` используется для объединение массива в строку.

```
let arrSlice = arr.slice(1, 3) // Срез, выводит содержимое массива с 1 по 3 индекс (два элемента)
arrSlice.join() // Собирает массив в строку: '2,3'
arrSlice.join(" - ") // '2 - 3'
```

Метод `match()` используется для поиска совпадений с регулярным выражением в строке. Он возвращает массив с найденными совпадениями или `null` , если совпадений не найдено.

```
let stringForRegex = "Текст для проверки текста"
stringForRegex.match(/текст/)[0] // Получить содержимое первого совпадения
stringForRegex.match(/текст/).index // Возвращает порядковый индекс первого совпадения в тексте
stringForRegex.match(/текст/i)[0] // Возвращает только первое совпадение без учета регистра
stringForRegex.match(/текст/gi) // Получить массив всех совпадений: [ 'Текст', 'текст' ]

stringForRegex = "2024-10-25"
stringForRegex.match(/(\d{4})-(\d{2})-(\d{2})/) // Группа захвата, которая возвращает полное совпадение
// [ '2024-10-25', '2024', '10', '25', index: 0, input: '25-10-2024', groups: undefined ]
```

Метод `search()` возвращает только индекс первого совпадения.

```
"Текст для проверки текста".search('про') // 10
```

Метод `replace()` заменяет найденные совпадения в строке на другие значения.

```
stringForRegex = "Текст для замены"
stringForRegex.replace(/для/, "после")           // 'Текст после замены'
stringForRegex.replace(/^т/i, "Этот т")          // 'Этот текст для замены'
stringForRegex.replace(/$/, "!")                  // 'Текст для замены!'
stringForRegex.replace(/(Текст)/, "$1 только")    // 'Текст только для замены'
stringForRegex.replace(/s[a-яА-Я]{3}/, "")        // 'Текст замены'
stringForRegex.replace(/s\p{L}+$/u, " кириллицы") // 'Текст для кириллицы'

stringForRegex = "Text for regex"
stringForRegex.replace(/s\w{3}/, "")              // Используется для замены любых латинских букв:
stringForRegex.replace(/s[a-zA-Z_]{3}/, "")        // эквивалент \w

stringForRegex = "2024-10-25"
stringForRegex = stringForRegex.replace(/(\d{4})-(\d{2})-(\d{2})/, "$3.$2.$1") // Поменять порядок
stringForRegex.replace(/d{2}\./g, "11.")      // Заменяет найденные две идущие цифры подряд: '11.:
stringForRegex.replace(/d{4}/, "2025")         // Заменяет четыре идущие цифры подряд: '25.10.2025
stringForRegex.replace(/20\d+/, "2025")        // Заменяет 20 и любые идущие за ним цифры: '25.10.:
stringForRegex.replace(/10.+\/g, "11.2025")    // Заменяет 10 и любое количество символов идущее за
stringForRegex.replace(/d{2,4}/g, "11")        // Заменяет найденные цифры следующие в порядке от :
stringForRegex.replace(/[45]/g, "1")           // Заменить 4 или 5 на 1: '21.10.2021'
```

Математические вычисления

```
Math.min(9, 10)      // Получить наименьшее значение двух чисел: 9
Math.max(9, 10)      // Получить максимальное значение двух чисел: 10
Math.floor(10 / 3)   // Округлить в меньшую сторону: 3
Math.ceil(10/3)      // Откруглить в большую сторону: 4
Math.trunc(4.9)      // Отбрасывание дробной части: 4
Math.round(4.5)      // Округление до ближайшего целого: 5
Math.round(4.45)     // Округление до ближайшего целого: 4
Math.fround(5.05)    // Ближайшее число с плавающей точкой одинарной точности: 5.0500001907341
Math.random()        // Псевдослучайное число между 0 и 1, например, 0.2309471255442206
Math.abs(-7)         // Получить абсолютное значение: 7
Math.sign(-3)        // Определение знака числа (-1, 0, 1): -1
Math.pow(2, 3)       // Возведение в степень: 8
Math.sqrt(16)        // Квадратный корень: 4
Math.cbrt(27)        // Кубический корень: 3
Math.imul(2, 4)      // Целочисленное 32-битное умножение: 8
Math.clz32(1)        // Количество ведущих нулей в 32-битном представлении: 31
```

Express

Создаем директорию, инициализируем проект и устанавливаем зависимости

```
mkdir api && cd api
npm init -y
npm install express
```

Серверная часть API сервера в файле `server.js`

```

const express = require('express')

const web = express()

// Middleware для парсинга JSON данных в теле запроса
web.use(express.json())

// Обработка GET запроса с параметрами в пути и заголовками
web.get('/user/:id', (req, res) => {
  const userId = req.params.id // Получаем параметр id из пути
  const customHeader = req.headers['custom-header'] // Чтение заголовка custom-header
  res.json({
    message: `GET запрос для пользователя с ID ${userId}`,
    customHeader: customHeader || 'Заголовок отсутствует'
  })
})

// Обработка POST запроса с данными в теле запроса и заголовками
web.post('/user', (req, res) => {
  const { name, age } = req.body // Получаем данные из тела POST запроса
  const authHeader = req.headers['authorization'] // Чтение содержимого из заголовка `authori
  if (authHeader !== 'Bearer TOKEN') {
    res.json({
      message: `Авторизация не пройдена, переданный токен: ${authHeader}`
    })
  } else {
    res.json({
      name: name || 'Не указано',
      age: age || 'Не указано',
    })
  }
})

// Запуск сервера на порту 3000
const PORT = 3000
web.listen(PORT, () => {
  console.log(`Сервер запущен на http://localhost:${PORT}`)
})

```

Запуск сервера

```
node server.js
```

Axios

Клиентская часть для работы с API

```
npm install axios
```

Пример GET запроса

```
const axios = require('axios')

// URL сервера
const url = 'http://localhost:3000'

// Пример GET запроса с параметром id в пути и кастомным заголовком
async function getUser(userId, Header) {
  try {
    const response = await axios.get(`${url}/user/${userId}`, {
      headers: {
        'custom-header': Header
      }
    })
    console.log('GET Ответ:', response.data)
  } catch (error) {
    console.error('Ошибка GET запроса:', error.message)
  }
}

await getUser(1, 'Value')
// GET Ответ: {
//   message: 'GET запрос для пользователя с ID 1',
//   customHeader: 'Value'
// }
```

Пример POST запроса

```
// Пример POST запроса с телом и заголовком авторизации
async function createUser(key, name, age) {
  try {
    const response = await axios.post(`${url}/user`, {
      name: name,
      age: age
    }, {
      headers: {
        'Authorization': `Bearer ${key}`
      }
    })
    console.log('POST Ответ:', response.data)
  } catch (error) {
    console.error('Ошибка POST запроса:', error.message)
  }
}

await createUser('KEY', 'Alex', 29)
// POST Ответ: { message: 'Авторизация не пройдена, переданный токен: Bearer KEY' }

await createUser('TOKEN', 'Alex', 29)
// POST Ответ: { name: 'Alex', age: 29 }
```


Fetch

```
async function fetchData(url) {
  try {
    // Отправляем GET-запрос на указанный URL
    const response = await fetch(url)
    // Проверяем, что ответ успешен
    if (!response.ok) {
      throw new Error(`Error Status: ${response.status}`)
    }
    // Получаем и выводим данные в формате JSON
    const data = await response.json()
    return data
  }
  // Обрабатываем ошибки
  catch (error) {
    console.error('Error:', error)
  }
}

const result = await fetchData('https://jsonplaceholder.typicode.com/todos/1')
result // { userId: 1, id: 1, title: 'delectus aut autem', completed: false }
JSON.stringify(result) // '{"userId":1,"id":1,"title":"delectus aut autem","completed":false}'
```

Cheerio

Cheerio - это библиотека для работы с HTML и XML в Node.js

```
npm install axios cheerio https-proxy-agent iconv-lite
```

Подключаем библиотеки и получаем содержимое страницы с помощью Axios через Proxy

```

const axios    = require('axios')
const cheerio  = require('cheerio')
const proxy    = require('https-proxy-agent')
const iconv    = require('iconv-lite')

// Имя агента в заголовке запросов (вместо axios)
const headers = {
  'User-Agent': 'Mozilla/5.0 (Windows NT 10.0 Win64 x64) AppleWebKit/537.36 (KHTML, like Gecko)

const proxyAddress = '192.168.3.100'
const proxyPort = 9090
const username = 'test'
const password = 'test'

// Создание экземпляра Axios с использованием конфигурации Proxy
const createAxiosProxy = () => {
  const config = {}
  config.httpsAgent = new proxy.HttpsProxyAgent(`http://${username}:${password}@${proxyAddress}`)
  return axios.create(config)
}

const axiosProxy = createAxiosProxy()

const url = "https://rutor.info"

// Отправляем запрос
const response = await axiosProxy.get(url, {
  responseType: 'arraybuffer',
  headers: headers
})

// Декодируем ответ
html = iconv.decode(response.data, 'utf8')

```

Вытаскиваем данные с помощью Cheerio

```

const data = cheerio.load(html)

// Обращаемся к элементу div (не обязательно указывать название элемента) с id="ws" > div с id='
data('div#ws #index tbody tr').length // 171
// Исключить из вывода строку с class="backgr" (заголовки столбцов)
data('#ws #index tbody tr').not('.backgr').length // 170

// Получить содержимое первого элемента "tr" в формате HTML, строки или текста без тегов
data('#ws #index tbody tr').not('.backgr').eq(0).html()
data('#ws #index tbody tr').not('.backgr').eq(0).toString()
data('#ws #index tbody tr').not('.backgr').eq(0).text()

// Получить содержимое элемента по частичному совпадению
data('#ws #index tbody tr').not('.backgr').find('td:contains("слово")').text().replace('\n', '')
// 'Мужское слово (2024) WEB-DLRip'

// Получить содержимое второго элемента по индексу в строке (столбце)
data('#ws #index tbody tr').not('.backgr').eq(0).find('td').eq(1).text().replace('\n', '').trim()
// 'Launcher for Zapret [v 1.3] (2024) PC | Portable'

// Получить содержимое атрибута "href" из элемента с классом "downgif"
data('#ws #index tbody tr').not('.backgr').eq(0).find('td a.downgif').attr('href')
// '//d.rutor.info/download/1008549'

// Получить содержимое атрибута "href" из второго элемента "a" в элементе "td"
data('#ws #index tbody tr').not('.backgr').eq(0).find('td a:nth-child(2)').attr('href')
// 'magnet:?xt=urn:btih:f1ca88b9421b243b6cb3d4da90e8fe133f381817&dn=rutor.info&tr=udp://opentor.

// Фильтруем все элементы по частичному совпадению
data('#ws #index tbody tr').not('.backgr').filter((index, element) => {
  // Проверяем, содержит ли текущая строка "tr" слово "Крит" в одном из столбцов "td"
  return data(element).find('td:contains("Крит")').length > 0
}).map((index, element) => {
  // Если слово найдено, извлекаем текст всех "td" в этой строке с помощью map()
  return data(element).find('td').map((index, element) => data(element).text()).get().join('
'}).get()

// [
//   '29 Окт 24 | Критик / The Critic (2023) WEB-DLRip 1080p от ExKinoRay | P | 6.10 GB | 1 4',
//   '28 Окт 24 | Критик / The Critic (2023) WEB-DLRip | P | 1.46 GB | 58 22',
//   '28 Окт 24 | Критик / The Critic (2023) WEB-DLRip-AVC от DoMiNo & селезень | P | 1.46 GB
//   '28 Окт 24 | Критик / The Critic (2023) WEB-DL 1080p | P | RGB | 1 | 5.63 GB | 115 54'
// ]

```

```

const torrents = []

// Собираем объект из всех элементов
data('#ws #index tbody tr').not('.backgr').each((index, element) => {
  const row = data(element)
  const torrent = {
    'Date': row.find('td').eq(0).text().trim(),
    'Name': row.find('td').eq(1).find('a').last().text().trim(),
    'Link': 'https://rutor.info' + row.find('td').eq(1).find('a[href^="/torrent"]').attr('href'),
    'DownloadLink': 'https://' + row.find('td').eq(1).find('a.downgif').attr('href'),
    'Magnet': row.find('td').eq(1).find('a[href^="magnet:"]').attr('href')
  }
  torrents.push(torrent)
})

// Конвертируем объект в формат JSON
console.log(JSON.stringify(torrents, null, 2))

// [
//   {
//     "Date": "29 Окт 24",
//     "Name": "Launcher for Zapret [v 1.3] (2024) PC | Portable",
//     "Link": "https://rutor.info/torrent/1008549/launcher-for-zapret-v-1.3-2024-pc-portable",
//     "DownloadLink": "https://d.rutor.info/download/1008549",
//     "Magnet": "magnet:?xt=urn:btih:f1ca88b9421b243b6cb3d4da90e8fe133f381817&dn=rutor.info&tr=
//   },
//   ...
// ]

```

Puppeteer

Puppeteer — это библиотека, которая предоставляет API для автоматизации любых действий в браузерах **Google Chrome** и **Mozilla Firefox** через протокол Chrome DevTools и WebDriver BiDi.

```
mkdir api && cd api && npm init -y && npm install puppeteer
```

Пример получения списка файлов раздачи с сайта [RuTor](https://rutor.info).

```

const puppeteer = require('puppeteer')
// Запускаем браузер и открываем новую пустую страницу
const browser = await puppeteer.launch({
  headless: true // Отключить отображение браузера (параметр по умолчанию)
})
const page = await browser.newPage()
// Открываем страницу с ожиданием загрузки 60 сек
const query = 721221
await page.goto(`https://rutor.info/torrent/${query}`, {
  timeout: 60000,
  waitUntil: 'domcontentloaded' // ожидать только полной загрузки DOM (не ждать загрузки внешн
})
await page.evaluate(() => {
  // Находим кнопку по JavaScript пути и нажимаем на нее
  // document.querySelector("#details > tbody > tr:nth-child(11) > td.header > span").click()
  // document.querySelector("#details > tbody > tr:nth-child(12) > td.header > span").click()
  // Находим все кнопки которые содержат class="button"
  const buttons = document.querySelectorAll('span.button')
  // Проходимся по найденным кнопкам
  buttons.forEach(button => {
    // Проверяем, содержит ли кнопка текст "Файлы" и нажимаем на нее
    if (button.textContent.includes('Файлы')) {
      button.click()
    }
  })
})
// Дождаться загрузки результатов
// const elementHandle = await page.waitForSelector('#files')
// Ищем элемент с идентификатором #files и проверяем, что элемент существует его содержимое не
await page.waitForFunction(() => {
  const element = document.querySelector('#files')
  return element && !element.textContent.includes("Происходит загрузка списка файлов...")
}, {
  timeout: 30000, // Ожидать результат 30 секунд
  polling: 50 // Проверка каждые 50мс (по умолчанию 100мс)
})
// Забираем результат после успешной проверки
const elementContent = await page.evaluate(() => {
  const element = document.querySelector('#files')
  return element ? element.textContent : null
})
// Закрываем браузер
await browser.close()

```

```

// Разбиваем полученные результаты на массив из строк (split) исключая первую строку (slice)
const lines = elementContent.trim().split('\n').slice(1)
// Регулярное выражение для разбиения строки на название и размер
const regex = /^(.+?)([\d.]+\s*\S+)\s+\((\d+)\)$/
const torrents = []
for (const line of lines) {
  const match = line.match(regex)
  const torrent = {
    'Name': match[1],
    'Size': match[2]
  }
  torrents.push(torrent)
}
console.log(JSON.stringify(torrents, null, 2))

```

Пример создания API для получения результатов проверки скорости интернета в формате JSON через [Ookla SpeedTest](#).

```

const puppeteer = require('puppeteer')
const browser = await puppeteer.launch({
  headless: false
})
const page = await browser.newPage()
await page.goto(`https://www.speedtest.net`, {
  waitUntil: 'domcontentloaded'
})
// Дождаться, когда кнопка "Go" станет доступной
await page.waitForSelector('span[data-testid="start-button"]')
// Возвращаем массив всех элементов span на странице с их текстом
await page.evaluate(() => {
  return Array.from(document.querySelectorAll('span')).map(element => ({
    Text: element.innerText, // Текст, отображаемый внутри элемента
    Content: element.textContent, // Весь текст внутри элемента, включая скрытые части
    // HTML: element.innerHTML, // HTML-код, который находится внутри элемента
    // AllHTML: element.outerHTML, // Весь HTML-код, который находится внутри элемента
    id: element.id, // Уникальный идентификатор элемента (#)
    className: element.className,
    tagName: element.tagName // Имя тега элемента (например, SPAN)
  }))
})
// Находим и нажимаем на кнопку
await page.evaluate(() => {
  const buttons = document.querySelectorAll('span')
  buttons.forEach(button => {
    if (button.textContent.includes('Go') || button.innerText.includes('GO')) {
      button.click()
    }
  })
})
// Функция для получения результата
async function checkResult () {
  return await page.evaluate(() => {
    const element = document.querySelector('div.result-data a')
    return element ? element.getAttribute('href') : null
  })
}
// Цикл для проверки получения результата
let resultUrl = '#'
// Проверяем, что результат содержит в начале строки 'http'
while (!resultUrl.startsWith('http')) {
  resultUrl = await checkResult()
}

```

```
}  
await browser.close()  
  
// Считываем данные из полученного url с помощью Fetch  
const result = await fetch(resultUrl)  
const resultHTML = await result.text()  
// Вытаскиваем JSON из HTML страницы  
const resultJSON = resultHTML.split('window.OOKLA')[3].replace('.INIT_DATA = ', '').replace(';\\r\\n', '')  
const resultObj = JSON.parse(resultJSON)  
  
resultObj.result.id // 16947429430  
resultObj.result.download // 7169 (7.17)  
resultObj.result.upload // 4939 (4.94)  
resultObj.result.idle_latency // 171 (ping)
```